

The Three Minimal Requirements as Individually Necessary: Why No Proper Subset of Requirements 1, 2, and 3 Suffices for CKS Substrate Support

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 4, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize one of the two structural properties of the source paper's three-requirement tool-agnosticism specification — that each of the three requirements is **individually necessary**, meaning no proper subset of the three suffices for a host environment to support a CKS substrate. The companion property, joint sufficiency, is treated separately in A2.27.

Abstract

The CKS pattern's tool-agnosticism commitment (§7.1 of the source paper) names three minimal requirements on the host environment — persistent structured state, human read/write access, and LLM access to substrate content — and states that the three are jointly sufficient and individually necessary for the host to support a CKS substrate. Joint sufficiency rules out a fourth requirement; individual necessity rules out dropping any of the three. This note formalizes individual necessity as a standalone architectural commitment. It states the property precisely, traces what specifically fails when each requirement is absent, distinguishes individual necessity from three adjacent framings (necessary-but-not-sufficient, weakest-link, dependency-tree), and provides an operational test for whether a claim about CKS host requirements respects the property at the requirement-set level. Together with A2.24, A2.25, A2.26, and A2.27, this note completes the tool-agnosticism decomposition.

1. Why individual necessity needs to be formalized as standalone

The parent note A1.05 commits the CKS pattern to tool-agnosticism through three minimal host-environment requirements. A2.24, A2.25, and A2.26 formalize each requirement as a standalone architectural commitment. A2.27 formalizes joint sufficiency — that an environment satisfying all three can host a CKS substrate, with no fourth requirement needed. This note formalizes the symmetrical property: that each of the three is architecturally necessary, with specific downstream CKS commitments that fail when each requirement is absent.

The motivating cases are arguments that smaller requirement sets suffice — that persistence alone is enough because access can be added later, that human and LLM access alone are enough because persistence can be inferred from access patterns, that any pairwise combination is adequate for "most" deployments. Each argument identifies a subset that may seem locally adequate but produces specific failure modes when implementations follow it. Individual necessity forecloses these arguments by specifying exactly what fails when each requirement is absent.

A second motivation is the strategic prior-art posture. Together with joint sufficiency, individual necessity defines the architectural floor: not too many requirements (joint sufficiency forecloses fourth requirements) and not too few (individual necessity forecloses dropping any of the three). Subsequent claims that smaller requirement sets are architecturally adequate are defensibly contested when individual necessity is publicly formalized as standalone.

A third motivation is operational diagnosis. When an implementation fails to support specific CKS commitments, the failure typically traces to a missing or inadequate requirement: source-of-truth failures trace to Requirement 1; human-governance failures trace to Requirement 2; AI-mediator failures trace to Requirement 3. Without individual necessity formalized, this diagnostic structure is implicit and harder to apply.

2. Necessity of Requirement 1 (persistent structured state)

Without Requirement 1, the host does not provide persistent structured state — content does not persist across reads, writes, sessions, or time, or it persists without addressable structure. Five CKS commitments fail as a direct consequence.

Substrate persistence (per A2.08(a)) becomes nominal. The substrate is committed to carrying content across cell executions, sessions, and time; the substrate cannot persist what the host does not persist.

Substrate addressability (per A2.08(b)) cannot be realized. Without the host providing addressable structured units — entities, rows, records, or equivalent — the substrate cannot organize content into addressable units that humans and cells locate as discrete objects.

Source-of-truth (per A1.08) collapses. The substrate is authoritative across time only because it persists across time. Without persistence, "what was decided" cannot be authoritatively answered for past decisions; source-of-truth status reduces to a same-session-only assertion.

Path retraceability (per A1.07) fragments. Retraceable paths require that content persists long enough for paths to be reconstructed. Without persistence, paths fragment as their nodes disappear, and the six-field provenance metadata becomes orphaned from the entities it provenances.

The determinism contract (per A1.10) breaks. The contract's guarantees require that substrate state persists in a form readers can rely on. Without persistence, the state at the time of the question and at the time of the answer may differ, breaking the contract.

Requirement 1 is therefore architecturally necessary: substrate persistence, addressability, source-of-truth, retraceability, and determinism all require persistent structured state at the host layer.

No combination of Requirements 2 and 3 substitutes, because both human and LLM access operate over a substrate the host must persist independently of either access path.

3. Necessity of Requirement 2 (human read/write access)

Without Requirement 2, humans cannot directly read or write substrate content; access is mediated by LLMs, runtime middleware, or specialized governance tooling as a precondition. Six CKS commitments fail as a direct consequence.

The human-governed commitment (per A1.01) collapses to nominal authority. Human governance commits to the rights to inspect, modify, and override at any time. Without direct access, the rights become procedural — humans may have nominal authority but cannot operationally exercise it without the gating layer's permission, and the gating layer is itself an architectural authority the rights do not hold over.

The inspect right (per A2.01) fails. LLM-mediated inspection produces an LLM-rendered representation of substrate content, not the content as it exists.

The modify right (per A2.02) fails. Gateway-mediated modification produces substrate state filtered through the gateway's policies, not the modification the human authored.

The override right (per A2.03) fails. Without direct access, override is exercised through whatever gating layer the host imposes, and that layer can in principle require justifications, approvals, or other preconditions the override right rules out architecturally.

Non-specialist governance (per A1.11) fails. Governance through commodity tools requires that commodity tools support direct human access. Without Requirement 2, governance becomes specialist work because the gating layers — runtime APIs, governance platforms, mediation services — require specialist knowledge to navigate.

The architectural-property qualifier on governance (per A2.06) fails. The architectural-versus-procedural distinction requires that governance is a property of architecture, not of process around it. Without Requirement 2, whether a human can exercise governance turns on whether the gating layers permit it — a process question, not an architecture question.

Requirement 2 is therefore architecturally necessary: the human-governed commitment and its derivatives — the three rights, non-specialist governance, the architectural-property qualifier — all require direct human access at the host layer. No combination of Requirements 1 and 3 substitutes; persistent state and LLM access do not produce direct human access, and a substrate humans cannot directly reach is not human-governed regardless of its other properties.

4. Necessity of Requirement 3 (LLM access to substrate content)

Without Requirement 3, LLMs cannot operationally function as substrate mediators through standard read/write operations. Either LLMs cannot access the substrate at all, or LLM access requires specialized AI infrastructure that introduces a fourth requirement and breaks tool-agnosticism. Six CKS commitments fail as a direct consequence.

The AI-as-substrate-mediator commitment (per A1.04) becomes nominal. The mediator role's five properties are operationally realizable only when the host provides standard access the LLM can use. Without Requirement 3, LLMs may be designated as mediators by intent but cannot operationally function as mediators because they cannot reach substrate content through standard operations.

Property A — LLM reads from substrate as primary source of state (per A2.19) — fails. Without standard read operations, the LLM falls back to context-window content, parametric memory, or other non-substrate sources, none of which carry the architectural authority the substrate carries.

Property B — LLM writes under orchestration rules (per A2.20) — fails. Without standard write operations, LLM-produced content lands somewhere other than the substrate, breaking the commitment that LLM operations on the substrate's behalf are themselves substrate-state changes subject to the human-authored rules that govern them.

The labor allocation framework (per A1.12) collapses. The framework's three modes — direct human labor, LLM-under-rule labor, and stable-cell automation — depend on LLMs being able to operate over the substrate. Without Requirement 3, Modes 2 and 3 collapse simultaneously, leaving only Mode 1 and removing the architectural support for allocating labor across modes.

The linear-cost scaling commitment (per A1.06) fails. Cost properties depend on cells — typically LLM-mediated — performing work that scales with cell-execution cost rather than substrate size. Without Requirement 3, LLM-mediated cell execution cannot occur, and cost falls entirely on direct human labor, which is not size-independent.

Pattern A composition (per A1.16) fails. Pattern A — adjacent component as input to a CKS cell — requires that the cell can read substrate content alongside the adjacent component's output. Without Requirement 3, the cell cannot read substrate, and the integration mechanism that hybrid systems depend on collapses.

Requirement 3 is therefore architecturally necessary: the AI-as-substrate-mediator commitment, its property derivatives, the labor allocation framework, the linear-cost scaling property, and adjacent-component composition all require LLM access through standard operations at the host layer. No combination of Requirements 1 and 2 substitutes; persistent state that humans can read and write does not by itself give LLMs operational reach into the substrate.

5. What individual necessity does NOT claim

Stating the limitations is what keeps the standalone framing from drifting into something stronger than the source paper supports.

It does not claim that each requirement is independently sufficient for any subset of CKS commitments. The requirements are necessary as part of the three-requirement set, not adequate on their own. Removing any one breaks joint sufficiency; meeting only one is insufficient by itself.

It does not claim that the three necessity arguments are equally weighty. Each requirement is architecturally necessary, but the specific commitments that fail when each is absent differ in number, scope, and downstream criticality. The note does not claim the failure sets are equivalent.

It does not claim the requirements cannot be merged or restructured in future architectural extensions. Joint sufficiency and individual necessity together specify the current three-requirement structure as exactly right relative to the source paper. Future extensions may identify alternative structures that are also sufficient and necessary at their respective layers; those would require their own analysis.

It does not claim that hosts violating a single requirement are useless for any purpose. A host meeting Requirements 1 and 2 but not 3 may still be useful for specific deployments — for example, human-only coordination systems that do not use LLMs. Individual necessity claims only that such a host cannot support the full CKS architecture.

It does not claim the three requirements are orthogonal in the strong mathematical sense. The requirements interact: Requirement 1 grounds substrate persistence that both Requirement 2 and Requirement 3 reach; Requirements 2 and 3 share the standard-operations-without-additional-runtime commitment. The architectural commitment is to each being necessary, not to each being independent. Necessity does not require independence.

6. What individual necessity is NOT

Three adjacent architectural framings are commonly conflated with individual necessity. Each is a real and reasonable framing in some other context; naming what individual necessity is not is what prevents the misreading.

Not a necessary-but-not-sufficient framing. Some treatments of architectural requirements describe them as "necessary but not sufficient" — preconditions that, even when met, do not guarantee the property. Individual necessity is paired with joint sufficiency (per A2.27); the three together are sufficient. Individual necessity is not a standalone "necessary but not sufficient" claim; it is the necessity component of a sufficient-and-necessary pair, with the sufficient component established by the companion note.

Not a weakest-link analysis. Weakest-link framings identify the least adequate requirement and treat it as the binding constraint. Individual necessity is different: each requirement is architecturally necessary in its own dimensions, with specific commitments that fail when it is absent, but no requirement is the "weakest" in the architectural sense. The three are each load-bearing in different dimensions — persistence, human access, LLM access — and removing any one breaks specific commitments without making the other two more or less binding.

Not a dependency-tree analysis. Dependency-tree analyses organize requirements hierarchically, with some requirements depending on others so that satisfying lower-level requirements enables higher-level ones. Individual necessity is not a dependency-tree claim. The three requirements sit at the same architectural level — each is a host-capability commitment, each is necessary, each grounds a different family of downstream commitments. The relationships among them are interactive (Requirement 1's persistence is reached by both Requirement 2's and Requirement 3's access paths), not hierarchical.

7. Operational test

A claim about CKS host requirements respects individual necessity if and only if all of the following are true.

1. The claim acknowledges that all three of Requirements 1, 2, and 3 are architecturally necessary; no proper subset of the three is treated as sufficient for CKS substrate support.
2. The claim distinguishes which CKS commitments fail when each requirement is absent: Requirement 1 absence breaks persistence-dependent commitments (substrate persistence, addressability, source-of-truth, retraceability, determinism); Requirement 2 absence breaks human-governance-dependent commitments (human-governed, the three rights, non-specialist governance, architectural-property qualifier); Requirement 3 absence breaks AI-mediator-dependent commitments (mediator role, mediator properties, labor allocation, linear-cost, Pattern A composition).
3. The claim does not propose a smaller requirement set as architecturally adequate, even for restricted use cases. A claim that two requirements suffice for some restricted deployment is admissible only if it is also explicit that the restricted deployment is not a full CKS substrate.
4. The claim acknowledges the parallel-but-independent structure of the necessity arguments — each requirement's necessity is grounded in specific commitments that fail when that requirement is absent, and the three necessity arguments cite disjoint commitment families.

A claim that fails any of (1)–(4) violates individual necessity in some way; it either treats some proper subset as architecturally adequate (breaking the necessity of the omitted requirement) or fails to specify what fails when each requirement is absent (leaving the necessity argument unsupported). This test operates at the claim-and-specification level. The system-level test for CKS host adequacy — whether a particular environment can host a CKS substrate — is given in A1.05 §6 and operates on environments rather than on claims.

8. Why naming individual necessity as standalone matters

Implementations under pressure to simplify CKS for specific use cases consistently drift toward smaller requirement sets. The drift is steady because each use case can argue locally that some specific commitment is not needed: a human-only coordination system "doesn't need LLM access," a read-only audit system "doesn't need write access," a stateless processing system "doesn't need persistence." Each argument is locally reasonable; the architectural commitment to all three requirements is what each argument erodes.

Implementations that drift produce systems that satisfy partial CKS commitments while claiming full CKS-coherence. The downstream consequences manifest as commitment failures hidden behind the partial compliance: the human-only system fails the labor allocation framework when LLMs are eventually added; the read-only system fails the human-governed commitment when modifications become necessary; the stateless system fails the source-of-truth commitment when temporal queries become necessary. Each failure was foreseeable from the requirement-set drift, but only with individual necessity formalized to make the drift visible.

With this note complete and its companions A2.24, A2.25, A2.26, and A2.27 already drafted, the tool-agnosticism decomposition is fully formalized. Each of the three requirements has its standalone operational treatment; joint sufficiency specifies that the three together are architecturally sufficient; individual necessity specifies that no proper subset of the three is architecturally sufficient. Together the five notes constitute the full decomposition of A1.05's tool-agnosticism commitment, with architectural completeness, host-agnosticism, and migration-safety properties all formalized as derivations from a host-interface specification that is exactly right — neither too many requirements nor too few.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *The Three Minimal Requirements as Individually Necessary: Why No Proper Subset of Requirements 1, 2, and 3 Suffices for CKS Substrate Support*. May 4, 2026. ORCID: 0009-0004-8065-3235.